

Report tecnico del codice

Progetto tank radiocomandato con Arduino Uno R4 WiFi, ESP32 controller, shield Emakefun PS2X & Motor Drive Board, due motori DC per cingoli, torretta orizzontale su driver esterno, servo di elevazione e relay sparo.

Data	13 luglio 2026
Repository	C:\Users\selmi\git\Embedded_Project_Tank_2025_2026
Tank	tank/ - firmware Arduino Uno R4 WiFi
Controller	controller-esp32/ - firmware ESP32 telecomando

1. Vista generale

Il progetto e' diviso in due firmware. Il tank crea una rete WiFi chiamata `Tank_AP` e ascolta pacchetti UDP sulla porta `4210`. Il controller ESP32 si collega a quella rete, legge i due joystick e i due pulsanti, poi invia continuamente un comando compatto al tank.

Joystick e pulsanti ESP32	Lettura ADC joystickReader	UDP V1 udpSender	Ricezione tank udpReceiver	Motori e servo main + moduli
-------------------------------------	--------------------------------------	----------------------------	--------------------------------------	--

La logica principale e' nel `main.cpp` del tank, ma il codice e' stato separato in moduli: cingoli in `trackController`, torretta ed elevazione in `servoTorretta`, rete in `udpReceiver` e libreria shield in `motorController/PWMController`.

2. Mappa hardware usata dal codice

Funzione	Collegamento	File dove si configura
Cingolo sinistro	Motore DC a 2 fili su shield <code>M3</code>	<code>tank/src/trackController.h</code>
Cingolo destro	Motore DC a 2 fili su shield <code>M1</code>	<code>tank/src/trackController.h</code>
Torretta orizzontale	Driver esterno: <code>D2=A/IN1</code> , <code>D3=B/IN2</code> , <code>D4=C/IN3</code> , <code>D5=D/IN4</code>	<code>tank/src/servoTorreta.h</code>
Elevazione torretta	Due servo accoppiati su shield <code>S5</code> e <code>S6</code>	<code>tank/src/servoTorreta.cpp</code>
Sparo	Relay su pin digitale <code>D7</code> . Sul modulo relay usa <code>COM</code> e <code>NO</code> , con <code>NO</code> verso il cannone.	<code>tank/src/main.cpp</code>
Joy1	Fisicamente <code>GPI034=X</code> e <code>GPI032=Y</code> . Poiche' il joystick e' montato ruotato, il software scambia gli assi: <code>driveX=GPI032</code> , <code>driveY=GPI034</code> .	<code>controller-esp32/src/joystickReader.cpp</code>
Joy2	Fisicamente <code>GPI035=X</code> e <code>GPI033=Y</code> . Dopo lo scambio: <code>turretX=GPI033</code> , <code>elevationY=GPI035</code> .	<code>controller-esp32/src/joystickReader.cpp</code>
Pulsanti	<code>GPI025</code> zero, <code>GPI026</code> fire, entrambi <code>INPUT_PULLUP</code>	<code>controller-esp32/src/joystickReader.cpp</code>

3. Protocollo UDP

Il formato mandato dal controller al tank e':

```
V1;driveX;driveY;turretX;elevationY;zero;fire
```

Campo	Valore	Uso nel tank
<code>driveX</code>	0-1023	Joy1 X: sterzo differenziale dei cingoli.
<code>driveY</code>	0-1023	Joy1 Y: avanti e indietro.
<code>turretX</code>	0-1023	Joy2 X: stepper torretta orizzontale su D2-D5.
<code>elevationY</code>	0-1023	Joy2 Y: servo elevazione su S5/S6.
<code>zero</code>	0 oppure 1	Azzera posizione logica torretta e servo.
<code>fire</code>	0 oppure 1	Genera impulso relay sparo.

Se il tank non riceve pacchetti per piu' di `500 ms` , il ricevitore mette tutti gli assi a `512` , spegne i pulsanti e marca `connected=false` . Nel `main.cpp` questo ferma i motori.

4. Firmware tank - Arduino Uno R4 WiFi

4.1 tank/platformio.ini

Definisce il progetto PlatformIO del tank: board `uno_r4_wifi`, piattaforma `renesas-ra`, framework `arduino` e monitor seriale a `9600` baud.

4.2 tank/src/main.cpp

Questo e' il vero centro del firmware. Non contiene piu' tutta la logica dei motori: inizializza i moduli, riceve i comandi e decide quando chiamare ogni sottosistema.

Parte	Cosa fa
Configurazione relay	Usa <code>FIRE_RELAY_PIN 7</code> , <code>FIRE_RELAY_ACTIVE_HIGH</code> e <code>FIRE_RELAY_PULSE_MS 200</code> . Il relay riceve un impulso breve quando arriva una pressione nuova del pulsante fire. Dopo ogni colpo il firmware aspetta <code>FIRE_RELAY_COOLDOWN_MS 12000</code> , quindi tra un colpo e l'altro passano 12 secondi.
Monitor seriale	Stampa coordinate e stato connessione ogni <code>1000 ms</code> , cosi la console non viene riempita a ogni pacchetto.
<code>MotorController shield(50)</code>	Oggetto che comanda la PCA9685 della shield a <code>50 Hz</code> , valore adatto ai servo e usato anche per i canali motore.
<code>configureShieldAddress()</code>	Cerca automaticamente la PCA9685 agli indirizzi <code>0x60</code> , <code>0x40</code> e <code>0x7F</code> . Se la trova, imposta l'indirizzo prima di inizializzare la shield.
<code>setFireRelay()</code>	Traduce lo stato logico acceso/spento nel livello elettrico giusto, rispettando se il relay e' active high o active low.
<code>updateFireRelay()</code>	Rileva il fronte di pressione del pulsante fire. Questo evita che il relay resti acceso finche' tieni premuto il pulsante e blocca nuovi colpi finche' non sono passati 12 secondi.
<code>stopAllMotion()</code>	Ferma i cingoli se la shield e' pronta e ferma sempre la torretta orizzontale su D2-D5.
<code>setup()</code>	Avvia seriale, relay, torretta diretta, I2C, shield, cingoli, servo elevazione e access point UDP.
<code>loop()</code>	Riceve un <code>TankCommand</code> , aggiorna torretta, cingoli, elevazione, relay, timeout, zero e stampa diagnostica.

4.3 tank/src/udpReceiver.h e udpReceiver.cpp

Questo modulo gestisce la rete sul tank. Il file header dichiara la struct `TankCommand`, cioe' il pacchetto gia' convertito in dati comodi per il resto del firmware.

Elemento	Spiegazione
WIFI_SSID e WIFI_PASS	Nome e password della rete creata dal tank: Tank_AP / 12345678 .
UDP_PORT 4210	Porta UDP su cui tank e controller comunicano.
CONNECTION_TIMEOUT_MS 500	Tempo massimo senza pacchetti prima di considerare perso il controller.
lastCommand	Ultimo comando valido ricevuto. Prima del primo pacchetto vale tutto centro e disconnesso.
UdpReceiver_begin()	Crea l'access point con WiFi.beginAP() , aspetta, poi avvia UDP.
UdpReceiver_update()	Legge il pacchetto, fa sscanf() sul protocollo V1, limita i valori tra 0 e 1023 e aggiorna lo stato connessione.

4.4 tank/src/trackController.h e trackController.cpp

Questo modulo muove i due cingoli. Ogni cingolo e' trattato come un motore DC a 2 fili: il sinistro usa M3 e il destro usa M1 .

Configurazione	Significato
DRIVE_JOYSTICK_CENTER 512	Centro degli assi joystick dopo la conversione ESP32 0-4095 verso 0-1023.
DRIVE_DEADZONE 100	Zona morta attorno al centro, per evitare che i cingoli si muovano da soli.
LEFT/RIGHT_TRACK_MIN_PWM 900	PWM minimo separato per i due motori quando il comando esce dalla deadzone.
LEFT/RIGHT_TRACK_MAX_PWM MAX_SPEED	PWM massimo separato; MAX_SPEED corrisponde a 4096 livelli PCA9685.
LEFT_TRACK_MOTOR	Associa il cingolo sinistro a M3 .
RIGHT_TRACK_MOTOR	Associa il cingolo destro a M1 .
Costanti INVERTED	Servono a correggere direzione senza riscrivere codice.
TRACK_TURN_MID_* 700 / 55	Fino al valore logico X 700, la sterzata cresce progressivamente fino al 55%.
TRACK_TURN_PRE_MAX_* 999 / 70	Tra 700 e 999, la sterzata cresce dal 55% al 70%.
TRACK_TURN_FULL_SPEED_JOYSTICK_VALUE 1000	Da 1000 fino al fondo corsa, il comando di sterzata raggiunge il 100%.

Nel .cpp il joystick viene convertito in PWM motore. Sotto la deadzone il motore viene frenato; fuori deadzone la velocita aumenta in base a quanto spingi il joystick.

Funzione	Cosa fa
<code>updateTrackMotor()</code>	Converte il comando del joystick in direzione e valore PWM del motore DC.
<code>centeredAxis()</code>	Trasforma un asse 0-1023 in un valore centrato attorno a zero.
<code>shapeTurnCommand()</code>	Applica la curva 55%/70%/100% soltanto a Joy1 X prima del mixing. Joy1 Y rimane lineare e puo' raggiungere la velocita massima avanti/indietro.
<code>TrackController_stop()</code>	Ferma entrambi i motori dei cingoli con <code>DCbrake()</code> .
<code>TrackController_update()</code>	Fa il mixing differenziale: sinistro = avanti + sterzo, destro = avanti - sterzo.

```
forward = driveY - 512
turn    = shapeTurnCommand(driveX - 512)
left    = forward + turn
right   = forward - turn
```

La curva e' applicata prima del mixing differenziale. Non esiste piu' una seconda riduzione globale della sterzata: 55%, 70% e 100% sono quindi i livelli effettivi del comando `turn` nei tre punti configurati.

4.5 `tank/src/servoTorretta.h` e `servoTorretta.cpp`

Questo modulo contiene due sistemi diversi: lo stepper orizzontale della torretta e i due servo di elevazione. La torretta orizzontale non usa piu' i canali `S1-S4` della PCA9685: usa direttamente i digitali `D2-D5`, come nel codice che funzionava bene sull'hardware.

Parte	Spiegazione
<code>STEPS_PER_REV 2048</code>	Usato per convertire gli step logici in gradi con <code>StepperTorretta_getAngle()</code> .
<code>TURRET_DRIVER_A/B/C/D_PIN</code>	Mappa driver torretta su <code>D2</code> , <code>D3</code> , <code>D4</code> , <code>D5</code> .
<code>TURRET_STEP_INTERVAL_MS 2</code>	Tempo minimo tra due step della torretta orizzontale. Aumentalo per rallentarla.
<code>TURRET_JOYSTICK_DEADZONE 200</code>	Zona morta del Joy2, condivisa tra torretta orizzontale ed elevazione.
<code>TURRET_MIN_ANGLE/MAX_ANGLE</code>	Limiti software della torretta: da <code>-180</code> a <code>180</code> gradi logici.
<code>ELEVATION_MIN/MAX_ANGLE</code>	Limite meccanico dei servo di elevazione: da <code>0</code> a <code>47</code> gradi.
<code>ELEVATION_MIRROR_BASE 90</code>	Il secondo servo fa il movimento specchiato: angolo servo B = <code>90</code> - angolo servo A .

La funzione `writeTurretStep()` usa la sequenza a una bobina alla volta, uguale al vecchio codice diretto Arduino:

```
fase 0: A HIGH, B/C/D LOW
fase 1: B HIGH, A/C/D LOW
fase 2: C HIGH, A/B/D LOW
fase 3: D HIGH, A/B/C LOW
```

`StepperTorretta_updateJoystick()` legge `turretX`. Se il valore e' sotto il centro meno deadzone fa `stepLeft()`; se e' sopra il centro piu' deadzone fa `stepRight()`; se e' al centro spegne le uscite con `StepperTorretta_stop()`.

Per l'elevazione, `ServoTorretta_updateJoystick()` cambia l'angolo a piccoli passi ogni `30 ms` e poi chiama `shield->servoPairTurn(S5, S6, angle, 90)`.

4.6 `tank/lib/motorController`

Questa e' la libreria wrapper della shield. Nasconde i dettagli dei canali PCA9685 e offre funzioni piu' semplici al firmware.

Funzione o costante	Cosa fa
<code>M1</code> - <code>M4</code>	Nomi logici delle quattro uscite motore della shield.
Canali <code>M*_DIRECTION_CHANNEL</code> e <code>M*_SPEED_CHANNEL</code>	Mappano ogni uscita motore sui canali reali della PCA9685 e dei driver TB6612.
<code>S1</code> - <code>S8</code>	Mappano i connettori servo della shield sui canali PCA9685 verificati per questa scheda.
<code>DCrun()</code>	Imposta velocita' PWM e direzione di una uscita motore.
<code>DCbrake()</code> / <code>DCbrakeAll()</code>	Spengono un motore o tutti i motori della shield.
<code>bipolarStepperStop()</code>	Spegne le due uscite motore che formano uno stepper bipolare.
<code>servoTurn()</code>	Converte un angolo 0-180 in impulso servo tra <code>110</code> e <code>500</code> tick PCA9685.
<code>servoPairTurn()</code>	Comanda due servo insieme, con il secondo specchiato attorno a una base.
<code>shieldDigitalWrite()</code>	Scriva un canale PCA9685 come digitale alto/basso. Rimane nella libreria, anche se la torretta ora usa D2-D5.

4.7 `tank/lib/PWMController`

Driver minimale per il chip PCA9685. Parla su I2C con `Wire` e scrive direttamente i registri del chip.

Funzione	Spiegazione
<code>begin()</code>	Avvia I2C, resetta <code>MODE1</code> e mette <code>MODE2=0x04</code> per uscita push-pull.
<code>setPWMFreq()</code>	Calcola il prescaler PCA9685 e lo scrive mentre il chip e' in sleep mode.
<code>setPWM()</code>	Scriva quattro byte per canale: tempo ON basso/alto e tempo OFF basso/alto.
<code>readRegister()</code> / <code>writeRegister()</code>	Funzioni base per leggere e scrivere registri via I2C.

5. Firmware controller - ESP32

5.1 controller-esp32/platformio.ini

Definisce il firmware ESP32 con piattaforma `espressif32`, board `esp32dev`, framework Arduino, monitor seriale a `115200` baud e upload a `921600`.

5.2 controller-esp32/src/main.cpp

Il main del controller e' volutamente semplice:

```
setup():
  Serial.begin(115200)
  JoystickReader_begin()
  UdpSender_begin()

loop():
  if non sono passati 10 ms: return
  data = JoystickReader_read()
  UdpSender_send(data)
```

Il timer basato su `millis()` usa `UDP_SEND_INTERVAL_MS 10`. Il controller manda quindi circa 100 pacchetti al secondo senza bloccare il loop con un `delay()` finale.

5.3 joystickReader.h e joystickReader.cpp

Questo modulo legge l'hardware fisico del controller e riempie la struct `ControllerData`.

Elemento	Cosa fa
<code>ControllerData</code>	Contiene <code>driveX</code> , <code>driveY</code> , <code>turretX</code> , <code>elevationY</code> , <code>zeroPressed</code> , <code>firePressed</code> .
<code>readRawJoystick()</code>	Legge <code>analogRead()</code> 0-4095 dell'ESP32 e lo rimappa a 0-1023, uguale al lato tank.
<code>JoystickReader_begin()</code>	Imposta i pulsanti come <code>INPUT_PULLUP</code> , configura l'ADC a 12 bit e avvia la calibrazione automatica dei quattro assi.
<code>calibrateJoysticks()</code>	Aspetta 800 ms, poi media 80 campioni per asse. Il centro reale di ogni joystick viene trasformato nel centro logico <code>512</code> .
<code>CONTROLLER_DEADZONE 80</code>	Elimina il rumore vicino al centro misurato durante l'avvio.
<code>DRIVE_SWAP_X_Y</code> e <code>TURRET_SWAP_X_Y</code>	Sono entrambi <code>true</code> : Joy1 e Joy2 scambiano gli assi fisici X/Y per correggere il montaggio ruotato del controller.
Costanti <code>*_INVERTED</code>	Permettono di cambiare il verso di un asse dopo lo scambio; attualmente sono tutte <code>false</code> .
<code>JoystickReader_read()</code>	Legge, calibra, scambia e se necessario inverte i quattro assi; poi legge i due pulsanti.

5.4 udpSender.h e udpSender.cpp

Questo modulo collega l'ESP32 alla rete del tank e spedisce i dati letti dal controller.

Elemento	Spiegazione
<code>UdpSender_begin()</code>	Configura la modalita' station, abilita l'auto-reconnect e avvia <code>WiFi.begin()</code> senza bloccare il firmware in attesa.
<code>ensureWifiConnected()</code>	Controlla la rete a ogni invio. Se manca, tenta la riconnessione ogni 3 secondi; quando ritorna, aggiorna il gateway e riapre UDP.
<code>configureUdpConnection()</code>	Legge il gateway come IP del tank, chiude l'eventuale socket vecchio e riapre la porta UDP 4210 dopo ogni nuova connessione.
<code>tankIP</code>	Default <code>192.168.4.1</code> , poi viene sostituito dal gateway se disponibile.
<code>UdpSender_send()</code>	Costruisce la stringa <code>V1;...</code> con <code>snprintf()</code> e la manda via UDP.
Stampa coordinate	Mostra un blocco leggibile <code>ESP32 TX UDP</code> una volta al secondo, non a ogni pacchetto.

6. Sicurezze e comportamenti importanti

Sistema	Protezione
UDP	Timeout tank a <code>500 ms</code> e riconnessione ESP32 non bloccante ogni <code>3 s</code> .
Cingoli	Deadzone a <code>100</code> , freno quando il joystick torna al centro e regolazioni PWM separate per M3/M1.
Controller	Calibrazione automatica dei quattro centri, deadzone 80 e assi X/Y scambiati per entrambi i joystick.
Torretta orizzontale	Limite software da <code>-180</code> a <code>180</code> gradi logici e stop quando il Joy2 X e' centrato.
Elevazione	Limite meccanico da <code>0</code> a <code>47</code> gradi per non forzare i servo.
Sparo	Impulso relay di <code>200 ms</code> , non continuo, con pausa obbligatoria di <code>12 s</code> tra due colpi.
Seriale	Stampe coordinate ogni <code>1000 ms</code> per evitare flooding della console.

7. Dove modificare le cose

Vuoi cambiare...	File	Costante/funzione
Velocita' cingoli	tank/src/trackController.h	LEFT/RIGHT_TRACK_MIN/MAX_PWM , LEFT/RIGHT_TRACK_PWM_PERCENT
Fasce della sterzata differenziale	tank/src/trackController.h	Costanti TRACK_TURN_MID_* , TRACK_TURN_PRE_MAX_* e TRACK_TURN_FULL_SPEED_*
Direzione joystick guida	controller- esp32/src/joystickReader.cpp	DRIVE_X_INVERTED , DRIVE_Y_INVERTED
Scambio X/Y dei joystick	controller- esp32/src/joystickReader.cpp	DRIVE_SWAP_X_Y , TURRET_SWAP_X_Y
Direzione cingolo singolo	tank/src/trackController.h	LEFT_TRACK_INVERTED , RIGHT_TRACK_INVERTED
Pin torretta D2-D5	tank/src/servoTorretta.h	TURRET_DRIVER_A/B/C/D_PIN
Velocita' torretta	tank/src/servoTorretta.h	TURRET_STEP_INTERVAL_MS
Limiti torretta	tank/src/servoTorretta.cpp	TURRET_MIN_ANGLE , TURRET_MAX_ANGLE
Limiti elevazione	tank/src/servoTorretta.cpp	ELEVATION_MIN_ANGLE , ELEVATION_MAX_ANGLE
Durata sparo	tank/src/main.cpp	FIRE_RELAY_PULSE_MS
Tempo tra due colpi	tank/src/main.cpp	FIRE_RELAY_COOLDOWN_MS
WiFi e UDP	tank/src/udpReceiver.cpp e controller-esp32/src/udpSender.cpp	WIFI_SSID , WIFI_PASS , UDP_PORT

8. Compilazione

Comandi PlatformIO usati per verificare i due firmware:

```
C:\Users\selmi\.platformio\env\Scripts\pio.exe run -d tank  
C:\Users\selmi\.platformio\env\Scripts\pio.exe run -d controller-esp32
```

Firmware	Esito ultimo build	Memoria
tank	SUCCESS	RAM 26.8%, Flash 23.9%
controller-esp32	SUCCESS	RAM 13.8%, Flash 56.7%

9. Esito della revisione completa

Tutti i sorgenti dei due firmware e delle librerie locali sono stati riletti e confrontati con pinout, protocollo e documentazione. Le due build PlatformIO terminano con **SUCCESS**. Cppcheck non segnala errori funzionali; rimane soltanto un suggerimento di stile sul ciclo che cerca l'indirizzo I2C. Clang-format segnala una sola interruzione di riga nel calcolo della curva di sterzata.

Controllo	Esito	Nota
Pin e periferiche	OK	D2-D5 , D7 , M3/M1 e S5/S6 non si sovrappongono.
Contratto UDP	OK	Sender e receiver usano lo stesso protocollo V1, porta 4210 e scala 0-1023.
Riconnessione controller	OK	L'ESP32 ritenta ogni 3 secondi senza bloccare il loop e riapre UDP dopo il collegamento.
Sicurezza perdita controller	OK	Dopo 500 ms il tank centra i comandi, disattiva i pulsanti e ferma i movimenti.
Limiti meccanici	OK	Torretta limitata a +/-180 gradi logici; elevazione limitata a 0-47 gradi.
Build firmware	SUCCESS	Tank e controller ESP32 compilano senza errori.

Punti non bloccanti da conoscere

Priorita'	Area	Comportamento
Media	Curva sterzata	Tra il valore X 999 e 1000 il comando passa dal 70% al 100% in un solo punto ADC. E' coerente con la configurazione richiesta, ma puo' produrre uno scatto vicino al fondo corsa.
Bassa	Seriale tank	La lunga riga diagnostica a 9600 baud puo' sospendere brevemente il loop una volta al secondo. I PWM DC restano attivi, ma gli step della torretta e gli aggiornamenti servo possono avere una breve pausa.
Bassa	Test automatici	Non sono presenti unit test per parsing UDP, curva differenziale e limiti. La build verifica la compilazione, mentre il comportamento finale deve essere provato sull'hardware.

10. Riassunto finale

Il codice attuale ha una struttura pulita: il `main.cpp` del tank coordina, ma la logica pratica e' separata in moduli. I cingoli sono gestiti dalla shield su M3 e M1 con mixing differenziale e curva dedicata 55%/70%/100%; la torretta orizzontale e' tornata sui digitali D2-D5 per pilotare bene il driver esterno; l'elevazione usa i servo su S5/S6 ; lo sparo usa il relay su D7 ; il controller ESP32 calibra i joystick, scambia gli assi X/Y montati ruotati, invia tutto via UDP ogni 10 ms e si riconnette automaticamente dopo una perdita WiFi.